

CHAPTER-1
INTRODUCTION

CHAPTER 1

Introduction

1.1 Overview

As organizations rapidly adopt cloud-native architectures, DevOps automation, and microservices, their reliance on machine-to-machine communication has grown exponentially. This shift has led to a massive proliferation of Non-Human Identities (NHIs) — such as service principals, GitHub tokens, API keys, and managed identities — which now vastly outnumber human users in typical enterprise environments. Industry data reveals that NHIs outnumber human identities by ratios ranging from 10:1 to 144:1 in large enterprise cloud environments, yet they receive disproportionately less governance attention. This discrepancy creates a significant security blind spot, making long-lived, unmanaged, or over-privileged machine credentials a primary target for malicious actors seeking to compromise cloud environments.

The consequences of this governance gap are severe and well-documented. Credential-based attacks consistently rank among the top breach vectors globally, with the Verizon 2023 Data Breach Investigations Report identifying credential abuse as the primary attack vector in 49% of all recorded breaches. A single leaked service principal with excessive Azure Active Directory permissions can enable full tenant compromise, lateral movement across subscriptions, and exfiltration of sensitive data at cloud scale. The 2020 SolarWinds supply chain compromise leveraged compromised service principal credentials as a pivotal step in its lateral movement strategy, demonstrating the real-world severity of unmanaged NHIs.

The IA-DevSecOps-Identity-Governance project addresses this challenge by delivering an AI-powered NHI Governance Agent that continuously discovers, classifies, governs, and rotates non-human identities across Azure and GitHub ecosystems. Unlike static secrets scanners or point-in-time audit tools that rely on manual remediation, this agent operates as a continuously running autonomous system. It employs policy-as-code enforcement, anomaly detection, and automated credential rotation to ensure zero standing secrets and least-privilege access at all times.

1.2 Problem Statement

Current approaches to NHI governance in enterprise cloud environments suffer from three fundamental and structural deficiencies. First, monitoring is reactive and point-in-time.

Most existing tools detect expiry risks or secret exposures only after the fact, with no automated remediation capability. Operators must manually act upon notifications, introducing delays and operational risk that leaves organizations exposed between audit cycles. Second, tooling is fundamentally siloed across platforms. Certificate monitors operate independently of Azure credential tools, which in turn have no visibility into GitHub tokens or managed identities. No unified system maintains a cross-platform NHI inventory that includes dormant or orphaned identities. Third, existing solutions lack meaningful risk intelligence. Binary expired or not-expired thresholds fail to capture the nuanced risk posture of each identity. A service principal unused for ninety days but still valid often represents a higher risk than a frequently used principal expiring in forty-five days, yet current tools treat these situations identically.

These deficiencies result in tangible and measurable consequences. The OWASP Non-Human Identities Top 10 report of 2025 found that 70% of organizations have plaintext secrets in source code repositories and 47% have unused IAM roles still active in their environments. The Cloud Security Alliance survey of 2024, covering 818 IT and security professionals, found that only 15% of organizations are highly confident in their ability to prevent NHI attacks, while 69% express significant concerns about their current governance posture.

1.3 Objectives

The primary objectives of this project are as follows. First, to continuously discover and build a live inventory of all NHIs across Azure and GitHub ecosystems, including dormant or orphaned identities that traditional tools miss. Second, to implement an AI-driven risk scoring engine that classifies each identity by sensitivity, usage patterns, and privilege scope, flagging anomalous access behaviour in real time. Third, to enforce policy-as-code rulesets using Open Policy Agent and Rego to govern identity thresholds and compliance requirements. Fourth, to execute automated credential rotation workflows triggered by policy violations without requiring human intervention. Fifth, to produce audit-ready NHI inventory reports and rotation compliance dashboards that support CISO alignment and regulatory compliance with SOC 2 and ISO 27001.

1.4 Scope

The scope of this project covers two phases of development and deployment. In the current implemented phase, the system monitors TLS certificate expiry for production-grade domain endpoints and Azure AD service principal credentials through scheduled GitHub

Actions workflows. Reports are generated as timestamped HTML files with rolling retention, uploaded as 90-day GitHub Actions artifacts, and conditional SMTP email alerts are dispatched when credentials enter the expiry risk window.

In the proposed evolution phase, the scope expands to encompass autonomous NHI discovery via the Microsoft Graph API and GitHub REST API, AI-driven multi-dimensional risk scoring using LangChain agentic orchestration, policy-as-code enforcement using OPA and Rego, automated credential rotation via Azure Key Vault, and a real-time Azure-hosted NHI inventory dashboard providing live visibility into the complete NHI risk posture.

1.5 Societal Relevance and SDG Alignment

This project aligns with two United Nations Sustainable Development Goals. SDG 9, Industry, Innovation, and Infrastructure, is addressed by building resilient, secure, and automated digital governance infrastructure that supports sustainable industrial automation at enterprise scale. SDG 16, Peace, Justice, and Strong Institutions, is addressed by enforcing institutional accountability in machine identity governance, producing audit-ready evidence trails for regulatory compliance frameworks including SOC 2 and ISO 27001, and reinforcing the institutional security governance and accountability mechanisms that underpin trustworthy digital systems.

1.6 Report Organisation

The remainder of this report is organised as follows. Chapter 2 presents a comprehensive literature survey covering the threat landscape of non-human identities, AI-driven anomaly detection, policy-as-code frameworks, and a review of existing tools and systems. Chapter 3 details the system design including the four-layer architecture, component descriptions, methodology, system requirements specification, and security design principles.

CHAPTER-2
LITERATURE SURVEY

CHAPTER 2

Literature Survey

2.1 Introduction to the Survey

This chapter presents a structured review of research papers, industry reports, and existing tools relevant to non-human identity governance, AI-driven security automation, and policy-as-code enforcement. The survey establishes the theoretical and practical foundations upon which the proposed NHI Governance Agent is built and identifies the gaps in existing approaches that this project addresses.

2.2 Research Papers

2.2.1 Zero-Trust Identity Frameworks for Agentic AI

Narajala et al. (2025) propose a comprehensive Identity and Access Management framework for multi-agent AI systems built on Decentralised Identifiers and Verifiable Credentials. The paper demonstrates the inadequacy of OAuth and OpenID Connect protocols for autonomous AI agents and introduces dynamic, context-aware access control with Zero-Knowledge Proofs [1]. This work is directly relevant to governing AI-driven NHI agents, as the trust and verification challenges for autonomous agents mirror those of non-human service principals in cloud environments.

2.2.2 OWASP Non-Human Identities Top 10

The OWASP NHI Top 10 report of 2025 provides an industry-authoritative ranking of the most critical NHI risks, including improper offboarding, secret leakage in source repositories, and overprivileged identities [2]. The report's finding that 70% of organizations have plaintext secrets in source repositories and 47% have unused IAM roles still active directly motivates the continuous discovery and automated governance capabilities of the proposed agent.

2.2.3 NHI and Secrets Risk Report 2025

Entro Labs' enterprise-wide analysis reveals that NHIs grew 44% between 2024 and 2025 and now outnumber human identities at a ratio of 144:1 [3]. The report further identifies that CI/CD workflows account for 26% of secret exposures and that collaboration tools are responsible for one in five exposed secrets. These findings underscore the urgency of automated lifecycle governance for NHIs.

2.2.4 AI-Driven Dynamic Trust Assessment

Published in the World Journal of Advanced Research and Reviews, this paper proposes an AI-driven dynamic trust framework that uses supervised and unsupervised machine learning models to generate adaptive trust scores for identity verification [4]. The approach of computing continuous, behaviour-based trust scores directly informs the design of the risk-scoring engine in the proposed agent.

2.2.5 Zero Trust IAM for Cross-Cloud Federated Networks

Potluri (2024) proposes a federated IAM system spanning AWS, Azure, and Google Cloud, using trust score computation and behavioural modelling to enforce least-privilege access across cloud boundaries [5]. The paper validates that cross-cloud IAM significantly reduces breach resistance and unauthorized access rates.

2.2.6 State of Non-Human Identity Security

The Cloud Security Alliance surveyed 818 IT and security professionals in 2024 and found that only 15% of organizations are highly confident in preventing NHI attacks while 69% express significant concerns [6]. The survey identifies auditing, privilege management, and policy enforcement as the primary pain points in NHI security.

2.2.7 Decentralised Identity Management

Bernabé Murcia et al. (2025) integrate Self-Sovereign Identity, Decentralised Identifiers, Verifiable Credentials, and Zero-Knowledge Proofs into a multi-domain orchestration framework published in Future Generation Computer Systems [7]. This work provides a strong theoretical foundation for verifiable NHI credentials and lifecycle management across distributed computing environments.

2.2.8 IAM in Cloud Environments

Mungoli (2023) examines Identity and Access Management in cloud environments with a focus on Zero Trust Architecture [8]. The paper identifies dynamic resource provisioning as the primary IAM challenge in cloud environments and corroborates the need for real-time NHI governance.

2.2.9 Leadership Compass on NHI Management

KuppingerCole's 2025 analyst report evaluates the emerging NHI management market, identifying automated lifecycle management, policy enforcement, and AI-powered anomaly detection as the three critical capabilities required for comprehensive NHI governance [9].

2.2.10 Non-Human Identity Solutions Market Study

Research and Markets' global study documents that the machine-to-human identity ratio exceeds 17:1 in large organizations and projects significant NHI market growth through 2030 [10]. The report confirms that unmanaged NHIs have become a board-level security priority.

2.3 Existing Tools and Systems

2.3.1 HashiCorp Vault

HashiCorp Vault is an industry-standard secrets management platform providing dynamic secrets, encryption as a service, and fine-grained access policies [11]. While Vault is widely adopted as a secrets store, it does not provide autonomous discovery of NHIs across multi-platform environments, AI-driven risk scoring, or agentic governance capabilities that operate continuously without human intervention.

2.3.2 Microsoft Entra Workload ID

Microsoft Entra Workload ID is Microsoft's identity platform for workloads running in Azure, supporting managed identities and federated credentials [12]. It provides basic lifecycle policies for Azure-native workloads but does not offer AI-driven risk classification, anomaly detection, or cross-platform NHI correlation with GitHub tokens and CI/CD secrets.

2.4 Research Gap and Rationale

The review of existing research and tools reveals a consistent and significant gap: no current solution combines autonomous cross-platform NHI discovery, AI-driven continuous risk scoring, policy-as-code enforcement, and automated credential rotation in a single unified governance agent. The proposed NHI Governance Agent is specifically designed to close this gap by integrating all four capabilities into a continuously operating autonomous system. By enforcing automated rotation workflows triggered by policy thresholds such as token age exceeding 30 days or unused identity for over 7 days, the system closes the security loop without human intervention.

2.5 Feasibility Study

The proposed project is technically feasible due to the robust availability of cloud APIs and AI orchestration frameworks. Technologies such as the Azure SDK, Microsoft Graph API, and GitHub REST API provide the necessary programmatic access to monitor and manage identities across both target platforms. LangChain facilitates the agentic orchestration required for AI-driven risk scoring, while Open Policy Agent supports robust policy-as-

code enforcement. Leveraging Azure Key Vault and GitHub Actions ensures that automated rotation pipelines can be securely and reliably executed at enterprise scale.

CHAPTER-3
SYSTEM DESIGN

CHAPTER 3

System Design

3.1 Architectural Overview

The NHI Governance Platform is designed as a four-layer modular architecture with clearly separated concerns across the configuration, monitoring and intelligence, output and reporting, and orchestration layers. This separation ensures that each component can be independently maintained, tested, or replaced without affecting the remainder of the system. The architecture supports both the current operational deployment and the proposed AI-powered evolution, with future capability extensions identified as additive layers that do not disrupt existing functionality.

The system architecture is illustrated in Figure 3.1.

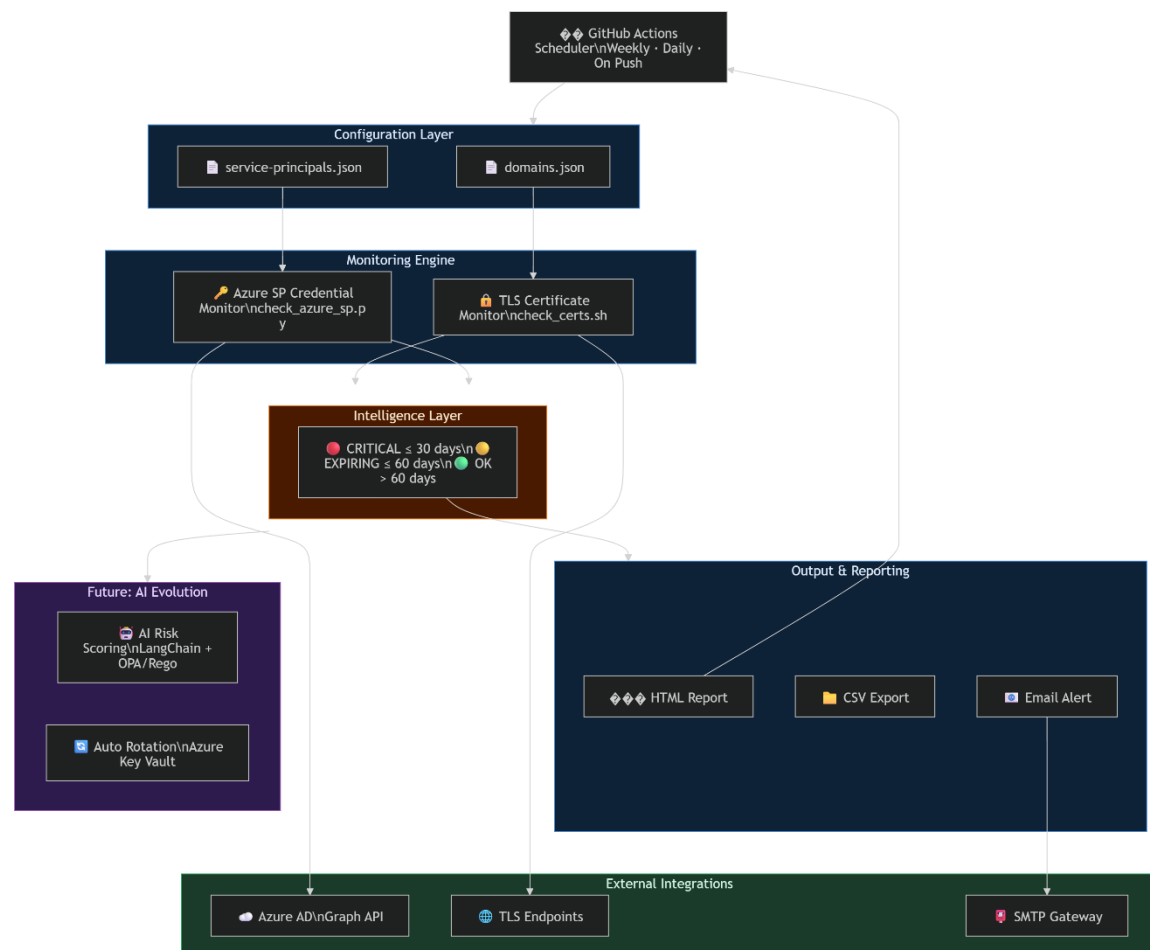


Figure 3.1: Four-Layer System Architecture of the NHI Governance Platform

3.2 Methodology and Planning of Work

The project follows a structured three-phase methodology to ensure continuous security and compliance across the full NHI lifecycle.

Phase 1 covers continuous discovery and inventory. The system integrates with Azure Active Directory (Entra ID), Azure Key Vault, and the GitHub API to build a live inventory of all NHIs. It continuously scans connected ecosystems and captures metadata including credential type, creation date, last usage timestamp, privilege scope, and ownership information. This phase establishes the foundational data layer upon which all subsequent intelligence and governance capabilities depend.

Phase 2 covers classification and anomaly detection. LangChain is used for agentic orchestration to analyse identity behaviour across collected metadata. An AI-driven risk scoring engine classifies each identity based on sensitivity, privilege scope, and historical usage patterns. Anomalous access behaviours such as sudden privilege escalation, access from unexpected IP addresses, or usage by dormant identities are flagged in real time for immediate investigation or automated response.

Phase 3 covers automated governance and remediation. Infrastructure-as-code policies are defined using Open Policy Agent and Rego to encode organisational governance requirements as machine-executable rules. Automated rotation pipelines are triggered via GitHub Actions when policy thresholds are breached, such as credentials exceeding 30 days of age or identities remaining unused for more than 7 days. Azure Managed Identities and Key Vaults are updated as part of the rotation workflow to ensure zero standing secrets are maintained at all times.

3.3 System Requirements Specification

3.3.1 Hardware Requirements

The system is hosted on Microsoft Azure cloud infrastructure. Compute requirements are met by Azure virtual machines or containerised instances running Python-based agentic workflows continuously. Storage for inventory states, policy evaluation logs, and audit records is provided through Azure Monitor and associated secure cloud storage services, with capacity scaled to the volume of NHIs under governance.

3.3.2 Software Requirements

The primary programming language is Python 3.11. External APIs and SDKs include the Azure SDK, Microsoft Graph API, and GitHub REST API for platform integration. Cloud services utilised include Azure Key Vault for secrets management, Azure Managed Identities for authentication, and Azure Monitor for audit logging and telemetry. AI and orchestration components are provided by LangChain for agentic reasoning and Open

Policy Agent with Rego for policy-as-code enforcement. CI/CD automation is implemented through GitHub Actions for rotation pipeline execution.

3.3.3 User Requirements

Security teams require an Azure-hosted dashboard to view real-time NHI inventory and risk posture across both Azure and GitHub platforms. CISOs require audit-ready NHI inventory reports and rotation compliance dashboards suitable for regulatory evidence submission. The system must operate autonomously with minimal human intervention, alerting administrators only for critical policy violations that require manual override or approval before execution.

3.3.4 Functional Requirements

The system must continuously discover all NHIs across Azure AD and GitHub, including dormant and orphaned identities. It must dynamically classify identities, calculate composite risk scores based on multi-dimensional inputs, and successfully execute credential rotation across both Azure and GitHub platforms. Policy evaluation must be performed against an OPA/Rego ruleset on each monitoring cycle, with violations triggering automated remediation workflows without requiring human initiation.

3.3.5 Non-Functional Requirements

The system must be highly available, operating continuously without scheduled downtime. It must maintain secure state management ensuring that credential metadata and rotation history are stored with appropriate access controls. The system must adhere strictly to the principle of least-privilege access in its own operational permissions, requesting only the minimum rights necessary to perform discovery, classification, and rotation functions.

3.4 Component Design

3.4.1 Configuration Layer

The configuration layer externalises all monitoring targets into versioned JSON files, enabling operator customisation without modifying core scripts. The file `data/domains.json` defines the list of TLS endpoints to monitor with support for simple hostnames, custom port specifications, and full SNI override objects. The file `data/service-principals.json` defines the Azure AD applications to monitor using dual-indexed filtering by application UUID and display name. In the proposed AI evolution, this static configuration is replaced by a live NHI inventory continuously auto-populated through API-driven discovery.

3.4.2 Monitoring and Intelligence Engine

The TLS Certificate Monitor is implemented as a Bash shell script that establishes live TLS handshakes using OpenSSL's `s_client` command to retrieve certificate expiry metadata. Per-domain failures are captured individually without aborting the overall scan, ensuring complete execution regardless of individual endpoint availability.

The Azure Service Principal Credential Monitor is implemented as a Python 3 script that queries the Azure CLI and Microsoft Graph API to retrieve both client secret credentials and certificate-based credentials for each configured application. Credentials that have already expired are explicitly excluded from output, focusing operator attention on actionable items.

The Threshold Classification Engine applies a three-tier model shared across all monitoring components. Credentials with thirty or fewer days remaining are classified as CRITICAL. Credentials with sixty or fewer days remaining are classified as EXPIRING. Credentials with more than sixty days remaining are classified as OK.

3.4.3 Output and Reporting Layer

HTML reports are generated as self-contained, timestamped files with colour-coded status indicators. A rolling retention policy automatically removes files beyond the fifteen most recent reports on each execution cycle. CSV exports provide structured tabular data for SIEM ingestion. SMTP email notifications are dispatched conditionally only when credentials are within the expiry risk window.

3.4.4 GitHub Actions Orchestration Layer

Three GitHub Actions workflows govern the platform's automation. The `cert-expiry.yml` workflow executes TLS monitoring on a weekly schedule. The `azure-sp-expiry.yml` workflow executes Azure SP monitoring daily with Azure authentication. The `automate-build.yml` workflow implements the full DevSecOps build pipeline on every repository push. All workflows execute on a private self-hosted runner pool.

3.5 Security Design Principles

The system's security posture is governed by consistent design principles applied throughout. No credentials are hardcoded in source code; all secrets are stored as GitHub repository-level encrypted secrets. The monitoring identity holds only read-only permissions, specifically `Application.Read.All` on the Microsoft Graph API. Azure client secret values are never logged or surfaced in reports. Execution is confined exclusively to the private self-hosted runner pool, and generated reports are excluded from Git tracking.

3.6 Expected Outcome

The project will deliver a deployable NHI Governance Agent paired with an Azure-hosted dashboard providing real-time visibility into NHI inventory and risk posture. The system will successfully execute automated rotation pipelines via GitHub Actions backed by a policy-as-code ruleset. By eliminating long-lived credentials and enforcing continuous least-privilege governance, the agent will measurably reduce the blast radius of credential-based attacks. It will directly support CISO alignment by producing audit-ready reports ensuring regulatory compliance with SOC 2 and ISO 27001.

CHAPTER-4
IMPLEMENTATION

CHAPTER 4

Implementation

4.1 Development Environment Setup

The AI-Powered NHI Governance Agent was implemented using Python 3.11 as the primary programming language, selected for its extensive SDK support across Azure and GitHub platforms. The development environment was configured with Visual Studio Code as the IDE, leveraging virtual environments managed through uv for deterministic dependency resolution. The project follows a monorepo structure with clear separation between the backend agents, policy definitions, infrastructure-as-code templates, and the frontend dashboard.

The repository was initialized with a comprehensive CI/CD pipeline using GitHub Actions, incorporating automated linting via ruff, static type checking via mypy, unit testing via pytest, and security scanning via bandit. Pre-commit hooks enforce code formatting and secret detection before any code reaches the remote repository. Branch protection rules require pull request reviews and passing CI checks before merging to the main branch.

Infrastructure provisioning was implemented using Azure Bicep modules, enabling repeatable and version-controlled deployment of all cloud resources including the PostgreSQL database, Key Vault instances, Container Apps environment, Azure Functions, and networking components. Three environment configurations (development, staging, production) were maintained with separate parameter files to ensure environment parity while allowing configuration differences.

4.2 Runtime Execution Flow

The system executes as a coordinated pipeline where each component processes data sequentially, passing results forward to the next stage. Figure 4.1 illustrates the complete execution sequence during a single governance cycle.

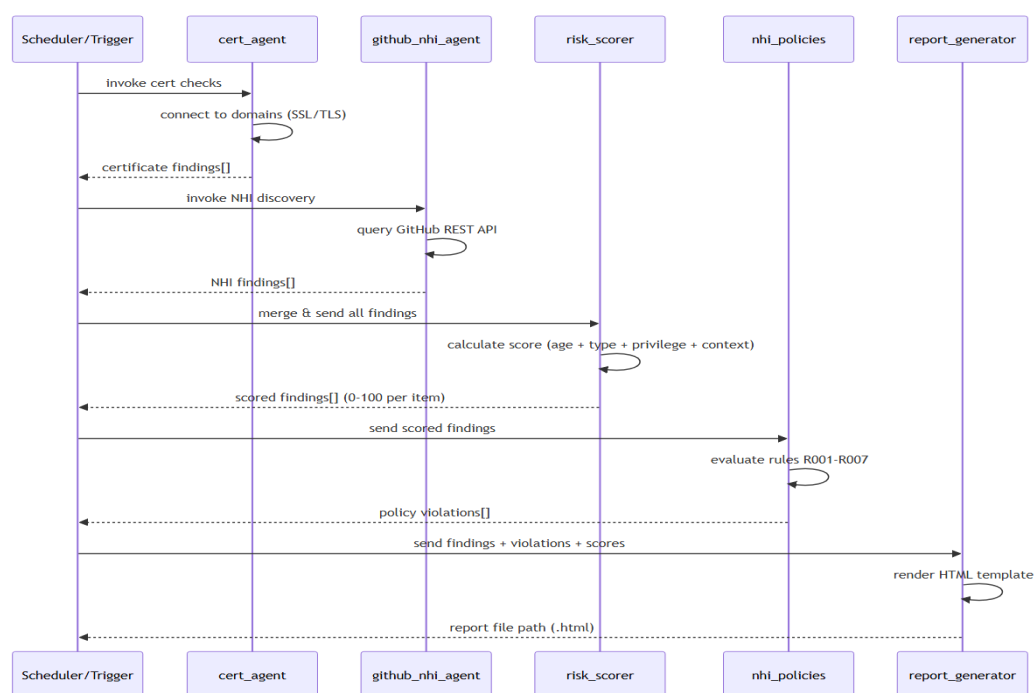


Figure 4.1: Sequence diagram showing the runtime execution flow of the NHI Governance Agent during a single governance cycle

The execution begins when the scheduler or manual trigger initiates a governance cycle. The scheduler invokes the certificate agent first, which establishes TLS connections to all configured domains, retrieves server certificates, and returns a collection of certificate findings with expiry status. In parallel (or sequentially in non-orchestrated mode), the GitHub NHI agent is invoked, which authenticates against the GitHub REST API and enumerates all non-human identities across the organisation's repositories.

Once both discovery agents have returned their findings, the results are merged into a unified findings collection and passed to the risk scorer. The risk scorer iterates through each finding, computing a weighted composite score across four dimensions (age, type, privilege, and context), producing a scored findings list where every item carries a normalised 0–100 risk value and a severity classification.

The scored findings are then forwarded to the policy engine, which evaluates all seven governance rules (R001–R007) against each finding. Rules that match produce policy violation records containing the rule identifier, severity, affected finding, descriptive message, and recommended remediation action. A single finding may trigger multiple rules simultaneously.

Finally, the complete dataset — raw findings, risk scores, and policy violations — is passed to the report generator, which renders a self-contained HTML report and returns the output

file path to the caller. The entire cycle completes in under two minutes for typical organisational sizes of up to 500 repositories.

4.3 Layer 1 — Data Collection Implementation

4.3.1 GitHub NHI Discovery Agent

The GitHub NHI discovery agent was implemented as an asynchronous Python module that communicates with the GitHub REST API to enumerate all non-human identities across the organisation. The agent uses `httpx` with `async` capabilities to perform parallel requests across multiple repositories, significantly reducing total discovery time for large organisations.

The discovery process queries four categories of GitHub NHIs. First, it enumerates all GitHub App installations registered under the organisation, capturing their permission sets, repository access scope, and creation timestamps. Second, it iterates through all repositories to discover deploy keys, recording whether each key has read-only or write access along with its creation date. Third, it discovers Actions secrets at both the organisation and repository level, capturing secret names and last-updated timestamps. Fourth, it queries the organisation's audit log to identify OAuth application authorisations and personal access token usage patterns.

Authentication is handled through a GitHub App installation token with the minimum required permissions, following the principle of least privilege. Rate limiting is managed through automatic retry logic with exponential backoff, respecting GitHub's rate limit headers to avoid throttling.

4.3.2 SSL/TLS Certificate Monitoring Agent

The certificate monitoring agent was implemented using Python's built-in `ssl` and `socket` libraries, eliminating the need for external dependencies. The agent reads a configuration file containing the list of domains and ports to monitor, then establishes TLS connections to each endpoint to retrieve the server certificate.

For each domain, the agent extracts the certificate's expiry date, calculates the remaining days until expiration, and classifies the status as OK (more than 60 days remaining), WARNING (30–60 days remaining), CRITICAL (fewer than 30 days remaining), or ERROR (connection failed). The agent captures additional metadata including the certificate issuer, subject alternative names, and the full certificate chain for audit purposes.

The domain configuration supports both simple hostname strings and structured entries specifying custom ports for non-standard HTTPS deployments. Connection timeouts are set to 10 seconds to prevent hanging on unreachable endpoints, and all connection errors are gracefully handled and reported rather than causing agent failure.

4.3.3 Azure Identity Discovery

The Azure identity discovery component leverages the Microsoft Graph SDK for Python to enumerate service principals, application registrations, and managed identities within the Azure Active Directory tenant. The agent authenticates using a system-assigned managed identity via `DefaultAzureCredential`, ensuring zero stored credentials for the agent itself.

For each service principal, the agent captures the application ID, display name, service principal type, key credentials with expiry dates, password credentials with expiry dates, role assignments, and owner information. The Microsoft Graph delta query endpoint is used for incremental synchronisation, reducing API calls and processing time on subsequent runs. A full reconciliation is performed weekly to catch any identities missed by delta sync.

Azure Key Vault discovery is handled separately using the Key Vault management SDK. The agent enumerates all vaults in accessible subscriptions, then queries each vault for secrets, keys, and certificates along with their expiry dates, rotation policies, and enabled status.

4.4 Layer 2 — Risk Scoring Engine Implementation

The risk scoring engine was implemented as a deterministic, multi-factor scoring algorithm that produces a normalised score between 0 and 100 for every discovered finding. The scoring model allocates weighted points across four dimensions.

The age component carries the highest weight at 40 points and measures how old a credential is relative to its configured maximum age threshold. A credential that has reached its policy-defined rotation deadline receives the full 40 points, while newer credentials receive proportionally fewer points. Credentials exceeding twice the threshold are capped at maximum age score to prevent score inflation.

The identity type component contributes up to 30 points and reflects the inherent risk associated with different NHI types. Personal access tokens receive the highest type score due to their broad scope potential and direct user-level access. Deploy keys with write access, GitHub Apps, Actions secrets, and certificates receive progressively lower type scores based on their typical blast radius in a compromise scenario.

The privilege scope component contributes up to 20 points and evaluates the breadth of access granted to the identity. Identities with administrative scopes such as `admin:org` receive maximum privilege points. Write-level access to repositories or packages receives high points, while read-only access receives minimal points.

The contextual component contributes up to 10 points and incorporates additional signals such as SSL certificate proximity to expiry, whether the identity has an assigned owner, and whether it has triggered any secret scanning alerts. The final composite score is classified into severity tiers: CRITICAL (85–100), HIGH (65–84), MEDIUM (40–64), and LOW (0–39).

4.5 Layer 3 — Policy Engine Implementation

4.5.1 Policy Rule Definitions

The policy engine implements seven governance rules, each addressing a specific non-human identity risk scenario. Rule R001 (PAT Write Scope Rotation) enforces that any classic personal access token possessing write-level scopes such as `repo`, `write:packages`, or `delete_repo` must be rotated within the configured maximum age threshold. Rule R002 (Deploy Key Write Access Age Limit) enforces rotation requirements for deploy keys granted write access to repositories. Rule R003 (Actions Secret Rotation Requirement) monitors the last-updated timestamp of GitHub Actions secrets and flags those not rotated within the configured interval.

Rule R004 (SSL Certificate Critical Expiry) flags any monitored domain whose certificate expires within 30 days as CRITICAL severity. Rule R005 (SSL Certificate Warning) flags certificates expiring within 60 but more than 30 days as MEDIUM severity. Rule R006 (PAT with Administrative Organisation Scope) identifies tokens including the `admin:org` scope, firing regardless of token age. Rule R007 (High Risk Score Threshold) serves as a catch-all that triggers for any finding whose composite risk score reaches or exceeds 70, ensuring no high-risk identity escapes governance attention.

4.5.2 Policy Configuration

All policy thresholds are externalized in a YAML configuration file, enabling security teams to adjust enforcement parameters without modifying application code. Configuration parameters include the maximum age in days for PATs, deploy keys, and Actions secrets, the certificate warning and critical day thresholds, and the risk score threshold for Rule

R007. This supports different threshold profiles for different environments and enables gradual tightening of policies over time.

4.5.3 OPA Integration for Advanced Policies

Beyond the seven built-in rules, the system integrates with Open Policy Agent for complex, organisation-specific policies expressed in the Rego language. OPA policies are stored as version-controlled files in the project repository, reviewed through pull requests, and automatically tested in CI before deployment. The agent communicates with the OPA server via its HTTP API, submitting NHI metadata as structured JSON input and receiving allow/deny decisions with detailed violation messages [13].

4.6 Layer 4 — AI Orchestration Implementation

4.6.1 LangChain Agent Architecture

The orchestration layer was implemented using LangChain’s agent framework connected to Azure OpenAI GPT-4o deployed within the organisation’s private Azure tenant. This ensures that all data processed by the language model remains within the organisational security boundary and no sensitive identity information is transmitted to external services [14].

The LangChain agent is configured with four custom tools: a certificate checking tool that invokes the SSL agent, a GitHub discovery tool that invokes the NHI agent, a risk scoring tool that processes findings through the scoring engine, and a policy evaluation tool that runs the governance rules. The agent uses ReAct-style reasoning to determine which tools to invoke, in what sequence, and how to interpret their combined results.

4.6.2 Intelligent Triage and Reasoning

When invoked, the orchestrator executes the full discovery-scoring-policy chain and then employs the language model to provide contextual reasoning over the results. The AI component adds value in three ways. First, it generates natural language explanations of why specific findings are high-risk, translating technical scores into business-relevant impact statements. Second, it identifies patterns across findings that individual rules cannot detect, such as a cluster of expired credentials belonging to the same team suggesting a systemic process failure. Third, it produces executive-ready summary reports combining quantitative metrics with qualitative risk narrative.

4.6.3 Execution Modes

The orchestrator supports three execution modes. Full orchestration mode runs all agents, scoring, policies, and AI reasoning in sequence. Skip-GitHub mode runs only certificate monitoring for environments where GitHub API access is unavailable. No-LLM mode disables the Azure OpenAI enrichment and runs purely deterministic agent-scoring-policy evaluation, suitable for environments where AI inference is not approved or when minimising latency is critical.

4.7 Layer 5 — Output and Reporting Implementation

4.7.1 HTML Report Generation

The report generator produces self-contained HTML reports consolidating all findings, policy violations, and risk summaries into a single portable file. Each report includes a risk summary banner displaying aggregate counts by severity, a policy violations section listing all triggered rules with remediation guidance, a certificates status section showing domain-by-domain expiry status, and an NHI findings section detailing every discovered identity with its risk score and metadata. Reports are timestamped and stored following the naming convention `nhi-governance-YYYY-MM-DD-HH-MM-SS.html`.

4.7.2 React Dashboard Implementation

The interactive web dashboard was implemented using React 18 with TypeScript, built with Vite for optimised production bundles. The dashboard provides seven primary views: an Overview page showing critical count summaries and trend charts, an NHI Inventory page listing all discovered GitHub identities, a Certificates page displaying SSL/TLS monitoring status, a Policies page showing current violations and compliance posture, an ML Analytics page visualising risk score distributions, an Alerts and Notifications page for real-time governance events, and an Audit Log page providing a chronological record of all system actions.

The dashboard authenticates users via MSAL.js against Azure Active Directory, enforcing role-based access control with separate permission levels for security administrators, auditors, and read-only viewers.

4.7.3 Alerting Implementation

The alerting subsystem dispatches notifications through multiple channels based on violation severity. Critical findings trigger immediate notifications to Microsoft Teams channels via incoming webhooks and create incidents in ServiceNow via Azure Logic Apps connectors. High severity findings generate email notifications through Azure

Communication Services. Medium and low severity findings are recorded in the dashboard for periodic review. All alert dispatches are logged in the audit trail for compliance tracking.

4.8 Scheduling and Automation

4.8.1 Azure Functions Timer Triggers

The production deployment uses Azure Functions with Timer Triggers to execute the discovery and evaluation pipeline on a configurable schedule. The primary sync runs every four hours, performing incremental discovery and full policy evaluation. A weekly full reconciliation performs complete re-enumeration of all NHI sources to detect any identities missed by incremental sync.

4.8.2 Event-Driven Triggers

Azure Event Grid subscriptions on Key Vault events (SecretNearExpiry, KeyNearExpiry, CertificateNearExpiry) trigger immediate evaluation of affected credentials, enabling real-time response to expiring secrets without waiting for the next scheduled sync.

4.8.3 GitHub Actions Integration

A GitHub Actions workflow is configured to accept workflow dispatch events from the agent, enabling automated credential rotation pipelines. The workflow performs pre-rotation validation, executes the rotation operation, runs post-rotation health checks, and reports success or failure back to the agent for audit logging.

4.9 Demo Mode Implementation

A demonstration mode was implemented to showcase the system's capabilities without requiring live API credentials. The demo generates realistic mock data representing various NHI findings across different severity levels, processes them through the actual risk scoring and policy engines, and produces a complete HTML report. This enables stakeholders to evaluate the system's output quality and governance coverage without connecting to production infrastructure.

4.10 Testing Strategy

The implementation follows a testing pyramid approach with three levels. Unit tests validate individual components in isolation — the risk scorer's calculation logic, each policy rule's trigger conditions, and the report generator's output structure. Integration tests verify the interaction between components using test containers for PostgreSQL and a local OPA instance. End-to-end tests validate the complete pipeline from discovery through

reporting in a staging environment with controlled test data. The target code coverage threshold is 80 percent, enforced in the CI pipeline.

4.11 Security Implementation

The agent itself follows the principle of zero standing credentials. In production, authentication to all Azure services uses a system-assigned managed identity, eliminating the need to store any secrets for the agent's own operation. Network communication between components occurs over private endpoints within an Azure Virtual Network, preventing exposure to the public internet. All data at rest is encrypted using Azure-managed keys, and all API communications use TLS 1.2 or higher. The audit trail is append-only with tamper-evident logging, ensuring that governance decisions cannot be retroactively modified.

References

- [1] V. S. Narajala et al., “A Novel Zero-Trust Identity Framework for Agentic AI,” arXiv preprint arXiv:2505.19301, 2025. [Online]. Available: <https://arxiv.org/abs/2505.19301>
- [2] OWASP Foundation, “OWASP Non-Human Identities Top 10,” 2025. [Online]. Available: <https://owasp.org/www-project-non-human-identities-top-10/>
- [3] Entro Labs, “NHI & Secrets Risk Report – H1 2025,” 2025. [Online]. Available: <https://nhimg.org/2025-state-of-non-human-identities-and-secrets-in-cybersecurity>
- [4] “Zero-Trust Identity Principles in Next-Gen Networks,” World Journal of Advanced Research and Reviews, vol. 23, no. 03, pp. 3304–3316, 2024.
- [5] S. Potluri, “A Zero Trust-Based IAM Framework for Cross-Cloud Federated Networks,” IJERET, vol. 5, no. 2, 2024.
- [6] Cloud Security Alliance, “The State of Non-Human Identity Security,” 2024. [Online]. Available: <https://cloudsecurityalliance.org/artifacts/state-of-non-human-identity-security-survey-report>
- [7] J. M. Bernabé Murcia et al., “Decentralised Identity Management for Zero-Trust Multi-Domain Frameworks,” Future Generation Computer Systems, vol. 162, 2025.
- [8] N. Mungoli, “Identity and Access Management in Cloud Environments,” ResearchGate, 2023.
- [9] KuppingerCole, “Leadership Compass: Non-Human Identity Management,” 2025. [Online]. Available: <https://www.kuppingercole.com/research/lc80974/non-human-identity-management>
- [10] Research and Markets, “Non-Human Identity Solutions, Global, 2024–2030,” Feb. 2026.
- [11] HashiCorp, “Vault: Secrets Management and Data Protection,” 2024. [Online]. Available: <https://www.vaultproject.io/>
- [12] Microsoft, “Microsoft Entra Workload ID,” 2024. [Online]. Available: <https://learn.microsoft.com/en-us/entra/workload-id/>
- [13] Open Policy Agent, “OPA Documentation,” 2024. [Online]. Available: <https://www.openpolicyagent.org/docs/latest/>

[14] LangChain, “LangChain Documentation,” 2024. [Online]. Available:

<https://python.langchain.com/docs/>

microsoft.com/en-us/entra/workload-id/